

Workflow API Implementation Complete

Summary

The Workflow APIs have been fully implemented according to the specification. This marks **Step 1 of the MCP Backend** completion.

What Was Implemented

1. Enhanced Schemas ([schemas.py](#))

-  **WorkflowStepDto** - Create step with order, name, tool_name, config
-  **WorkflowStepResponse** - Step response with ID
-  **CreateWorkflowDto** - Create workflow with target_platform
-  **UpdateWorkflowDto** - Update workflow fields
-  **WorkflowListResponse** - List view with last_run_at/status
-  **WorkflowResponse** - Detail view with full steps
-  **RunWorkflowResponse** - /run endpoint response

2. Complete CRUD Endpoints ([workflow.py](#))

GET /api/workflows?workspace_id={id}

- Lists all workflows for workspace
- Returns summary view without steps
- Includes last run information
- Excludes soft-deleted workflows

GET /api/workflows/{id}

- Returns full workflow details
- Includes all steps with proper structure
- Returns 404 if not found or deleted

POST /api/workflows

- Creates new workflow
- Validates steps (length > 0, sequential order, valid tools)
- Returns created workflow with transformed steps

PUT /api/workflows/{id}

- Updates workflow metadata and/or steps
- Prevents updates while jobs are running
- Full replace of steps (not merge)
- Returns updated workflow

DELETE /api/workflows/{id}

- Soft delete (sets deleted_at)
- Prevents deletion while jobs are running
- Returns 204 No Content

POST /api/workflows/{id}/run ★ MOST IMPORTANT

- Triggers workflow execution
- Creates Job with status=pending
- Returns immediately (no inline execution)
- Returns job_id for polling

3. Validation Layer

Tool Registry

```
AVAILABLE_TOOLS = {  
    "blog_generator",  
    "image_generator",  
    "caption_generator",  
    "content_optimizer",  
    "hashtag_generator"  
}
```

Validation Rules

- ☒ Steps must exist (length > 0)
- ☒ Orders must be sequential (1, 2, 3...)
- ☒ Tool names must be registered
- ☒ Config must be valid JSON (Pydantic)
- ☒ Workspace isolation enforced
- ☒ Cannot update/delete with active jobs

Helper Functions

- `validate_workflow_steps()` - Validates step structure
- `check_active_jobs()` - Checks for running/pending jobs
- `transform_workflow_response()` - Formats DB data for API

4. Enhanced Job Router ([job.py](#))

GET /api/jobs/{job_id}

- Get job status and logs
- Frontend polls this for execution progress

GET /api/jobs/workflow/{workflow_id}

- Get all jobs for a workflow
- Supports filtering by status
- Supports limiting results
- Shows execution history

5. Database Migration (001_add_workflow_fields.sql)

- Adds `target_platform` column
- Adds `updated_at` column with auto-update trigger
- Adds `deleted_at` column for soft deletes
- Creates indexes for performance
- Adds documentation comments

API Examples

Create Workflow

```
POST /api/workflows
{
  "workspace_id": "ws_123",
  "name": "LinkedIn Blog Pipeline",
  "description": "Generate blog + image",
  "target_platform": "linkedin",
  "steps": [
    {
      "order": 1,
      "name": "Generate Blog",
      "tool_name": "blog_generator",
      "config": {
        "topic": "AI trends",
        "length": "medium"
      }
    },
    {
      "order": 2,
      "name": "Generate Image",
      "tool_name": "image_generator",
      "config": {
        "style": "professional"
      }
    }
  ]
}
```

List Workflows

```
GET /api/workflows?workspace_id=ws_123
```

Response:

```
[
  {
    "id": "wf_001",
    "name": "LinkedIn Blog Pipeline",
    "description": "Generate blog + image",
    "target_platform": "linkedin",
    "last_run_at": "2025-12-25T10:30:00Z",
    "last_run_status": "completed",
    "created_at": "2025-12-20T08:00:00Z"
  }
]
```

Run Workflow

```
POST /api/workflows/wf_001/run
```

Response:

```
{
  "job_id": "job_abc123",
  "workflow_id": "wf_001",
  "status": "pending",
  "started_at": "2025-12-26T12:00:00Z"
}
```

Poll Job Status

```
GET /api/jobs/job_abc123
```

Response:

```
{
  "id": "job_abc123",
  "workflow_id": "wf_001",
  "status": "pending", # or "running", "completed", "failed"
  "logs": {
    "message": "Job created and queued for execution",
    "workflow_name": "LinkedIn Blog Pipeline",
    "step_count": 2
  },
  "started_at": "2025-12-26T12:00:00Z",
  "finished_at": null,
  "created_at": "2025-12-26T12:00:00Z"
}
```

What Workflow APIs Do NOT Do ☒

- ☒ Execute tools directly
- ☒ Call AI models inline
- ☒ Return generated content
- ☒ Handle posting
- ☒ Handle analytics

All execution happens via the Job processor (to be implemented next).

Definition of DONE Checklist

- ☒ **Workflow DB table** - Enhanced with new columns
- ☒ **CRUD APIs** - Complete implementation
- ☒ **Validation layer** - Tool registry + business rules
- ☒ **/run endpoint** - Creates Job, returns immediately
- ☒ **No inline AI execution** - Jobs are queued only

Next Steps (Not Implemented Yet)

1. **Apply database migration** - Run `001_add_workflow_fields.sql` on Supabase
2. **Job Execution Engine** - Background processor to execute workflow steps
3. **Tool Implementations** - Actual blog_generator, image_generator logic
4. **Queue System** - RabbitMQ/Celery for async job processing
5. **Frontend Integration** - Remove mock data, connect to real APIs

Testing the APIs

Start the server:

```
cd backend-python
python -m uvicorn main:app --reload
```

Test endpoints:

```
# Create workflow
curl -X POST http://localhost:8000/api/workflows \
  -H "Content-Type: application/json" \
  -d '{...}'

# List workflows
curl http://localhost:8000/api/workflows?workspace_id=xxx

# Run workflow
curl -X POST http://localhost:8000/api/workflows/{id}/run
```

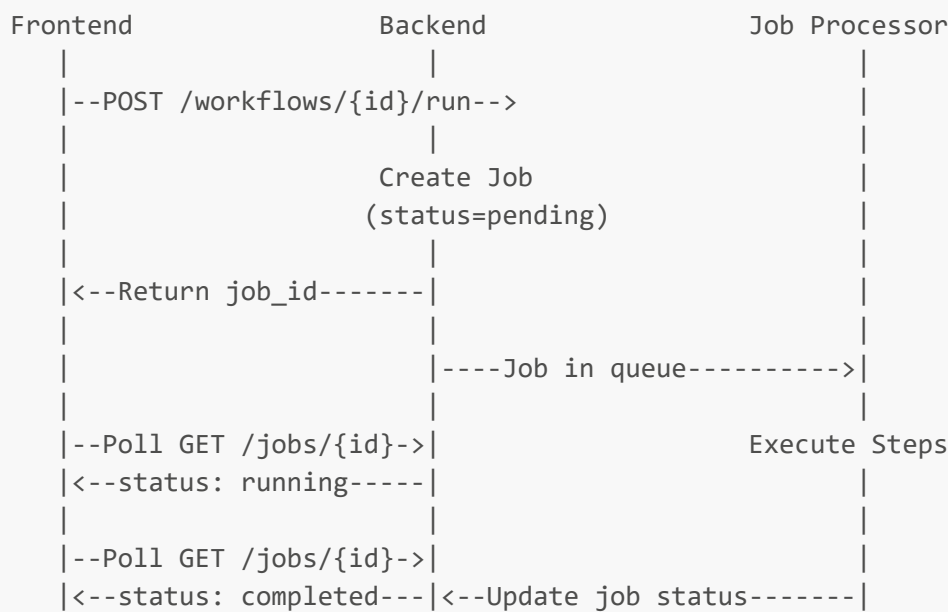
```
# Check job status
curl http://localhost:8000/api/jobs/{job_id}
```

Architecture Notes

Separation of Concerns

- **Workflows** = Definitions (WHAT to run)
- **Jobs** = Executions (WHAT is running)
- **Tools** = Implementations (HOW to run)

Async Execution Pattern



Files Modified

1. ☒ `app/schemas.py` - Enhanced workflow and job schemas
2. ☒ `app/routers/workflow.py` - Complete CRUD + validation + /run
3. ☒ `app/routers/job.py` - Enhanced job queries
4. ☒ `migrations/001_add_workflow_fields.sql` - Database schema

Congratulations! 🎉

The MCP backbone officially exists.

Workflows can now be:

- Created and stored in DB
- Updated with validation
- Soft deleted with safety checks
- Triggered to create Jobs

- Tracked via Job polling

The foundation is solid. Ready for Step 2: Job Execution Engine.